



MEDIACREST
TRAINING COLLEGE

Machine Learning II: Classification Foundations & Baseline Models

Week 5, Day 2: Transitioning to Classification, Baseline
Algorithms & Evaluation Metrics

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

9th September 2026

Abstract

This presentation bridges the gap between regression and classification, establishing robust baseline models and mastering fundamental evaluation metrics using a real-world healthcare dataset.

- **Foundations:** Understanding the classification landscape, probability vs. class labels, and the Kenyan healthcare context.
- **Baseline Algorithms:** Deep dive into Logistic Regression and k-Nearest Neighbors (k-NN), including distance metrics and the curse of dimensionality.
- **Evaluation Metrics:** Why accuracy fails in imbalanced data, and mastering the Confusion Matrix, Precision, Recall, and F1-Score.
- **Application:** Hands-on Python lab predicting Heart Disease risk (`Heart.csv`), aligning with Week 5's healthcare practical applications.

Session Objectives

- Differentiate clearly between regression and classification tasks.
- Understand how probability thresholds translate into discrete class labels.
- Build and interpret Logistic Regression and k-NN baseline classifiers.
- Identify the pitfalls of using "Accuracy" as a sole metric in imbalanced datasets.
- Construct and interpret Confusion Matrices (TP, FP, TN, FN).
- Calculate and strategically optimize Precision, Recall, and F1-Score for policy and clinical decisions.
- Execute a complete Python classification pipeline on the `Heart.csv` dataset.

Part 1: The Classification Landscape

"From predicting quantities to predicting categories."

Regression vs. Classification

Regression

- **Output:** Continuous numerical value.
- **Goal:** Predict a quantity (e.g., Blood Pressure, House Price).
- **Metrics:** RMSE, MAE, R^2 .
- **Example:** Predicting a patient's exact cholesterol level.

Classification

- **Output:** Discrete category or class label.
- **Goal:** Predict a state or group (e.g., Disease Yes/No, Churn Yes/No).
- **Metrics:** Accuracy, Precision, Recall, F1, ROC-AUC.
- **Example:** Predicting if a patient has Heart Disease (AHD).

Probability vs. Class Labels

- Machine learning classifiers typically output a **probability** (e.g., 0.78 chance of Heart Disease), not a hard label immediately.
- A **decision threshold** (default is usually 0.50) is applied to convert this probability into a discrete class label:
 - If $P(\text{Class} = 1) \geq 0.50 \rightarrow \text{Predict Yes (1)}$
 - If $P(\text{Class} = 1) < 0.50 \rightarrow \text{Predict No (0)}$
- **Critical Insight:** The 0.50 threshold is arbitrary. In healthcare (e.g., MOH policy), we may lower the threshold to 0.30 to catch more potential cases (increasing Recall), accepting more False Positives.

The Kenyan Healthcare Context

Practical Application: Heart Disease Risk Prediction

While the curriculum mentions Telecom Churn, today's hands-on lab utilizes the `Heart.csv` dataset to demonstrate these exact classification principles within the **Kenyan Healthcare Context**.

- **Scenario:** County health facilities need a triage tool to identify patients at high risk of heart disease based on routine vitals (Age, RestBP, Chol, MaxHR).
- **Policy Impact:** Accurate classification allows for targeted resource allocation, early intervention, and reduced strain on referral hospitals (e.g., KNH, MTRH).
- **Dataset:** `Heart.csv` containing clinical measurements and the target variable HD (Heart Disease: 0 = No, 1 = Yes).

Part 2: Baseline Algorithms

"Establishing robust, interpretable starting points."

Logistic Regression Revisited

- Despite the name, Logistic Regression is a **classification** algorithm, not a regression algorithm.
- It models the log-odds of the probability of a binary outcome as a linear combination of features.
- **Why use it as a baseline?**
 - Highly interpretable (coefficients map to Odds Ratios).
 - Computationally efficient.
 - Provides well-calibrated probabilities.
 - Serves as a "sanity check" before deploying complex black-box models.

The Sigmoid Function

- Logistic Regression uses the **Sigmoid (Logistic) Function** to squash any real-valued number into a range between 0 and 1.
- Formula:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$.

- The output $\sigma(z)$ is interpreted directly as the estimated probability $P(Y = 1|X)$.

Note

The decision boundary is linear in the feature space, meaning Logistic Regression is a **linear classifier**.

k-Nearest Neighbors (k-NN) Concept

- **Type:** Instance-based, non-parametric, lazy learning algorithm.
- **Mechanism:** To classify a new data point, the algorithm finds the k closest data points in the training set and assigns the majority class among those k neighbors.
- **Key Hyperparameter:** k (the number of neighbors).
 - Small k (e.g., $k = 1$): High variance, prone to overfitting (noisy decision boundaries).
 - Large k : High bias, smoother decision boundaries, may underfit.

k-NN: Distance Metrics

- k-NN relies entirely on measuring "distance" between data points.
- **Euclidean Distance (L2):** Straight-line distance. Most common.

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

- **Manhattan Distance (L1):** Sum of absolute differences (grid-like path). More robust to outliers than Euclidean.

$$d(p, q) = \sum_{i=1}^n |q_i - p_i|$$

- **Crucial Requirement:** Because distance is magnitude-dependent, **feature scaling (Standardization)** is **absolutely mandatory** for k-NN.

The Curse of Dimensionality

The k-NN Killer

As the number of features (dimensions) increases, the volume of the space increases so fast that the available data become sparse.

- In high-dimensional space, the distance between the "nearest" neighbor and the "farthest" neighbor converges to the same value.
- **Result:** Distance metrics lose their meaning, and k-NN performance degrades significantly.
- **Mitigation:**
 - ① Dimensionality reduction (PCA).
 - ② Rigorous feature selection (dropping irrelevant clinical variables).
 - ③ Using algorithms less sensitive to high dimensions (e.g., Tree-based models, covered in Day 2).

Part 3: Evaluation Metrics Deep-Dive

"Beyond Accuracy: Measuring what truly matters."

Why Accuracy is Misleading

- **Accuracy** = $\frac{\text{Correct Predictions}}{\text{Total Predictions}}$
- **The Imbalance Trap:** Imagine a dataset where only 5% of patients have Heart Disease (Positive class).
- A "dumb" model that **always predicts "No"** will achieve **95% Accuracy**.
- **Business/Clinical Reality:** This model is utterly useless because it fails to identify any actual disease cases (0% Recall).
- **Conclusion:** Accuracy is a dangerous metric for imbalanced datasets. We must look deeper.

The Confusion Matrix Defined

- A table that summarizes the performance of a classification model by comparing actual vs. predicted labels.
- It breaks down predictions into four fundamental outcomes:

Actual \ Predicted	Predicted: No (0)	Predicted: Yes (1)
Actual: No (0)	True Negative (TN)	False Positive (FP)
Actual: Yes (1)	False Negative (FN)	True Positive (TP)

True Positives & True Negatives

True Positive (TP)

The model predicted YES, and it was actually YES.

Healthcare Context: The model correctly identified a patient who truly has heart disease.

This is a WIN.

True Negative (TN)

The model predicted NO, and it was actually NO.

Healthcare Context: The model correctly cleared a healthy patient, avoiding unnecessary stress and tests.

This is a WIN.

False Positives & False Negatives

False Positive (FP) (Type I Error)

The model predicted YES, but it was actually NO.

Healthcare Context: A healthy patient is flagged as high-risk.

Cost: Unnecessary anxiety, extra clinical tests, wasted resources.

False Negative (FN) (Type II Error)

The model predicted NO, but it was actually YES.

Healthcare Context: A sick patient is sent home untreated.

Cost: Severe health deterioration, mortality, medical liability.

Precision

- **Definition:** Out of all instances the model **predicted as Positive**, how many were **actually Positive**?
- **Formula:** $\text{Precision} = \frac{TP}{TP+FP}$
- **When to Optimize:** When the cost of a **False Positive** is very high.
- **Example:** Spam email detection. You'd rather a spam email reach the inbox (FN) than an important client email go to spam (FP).

Healthcare Nuance

In heart disease screening, we typically *sacrifice* some Precision to gain Recall, because missing a sick patient (FN) is worse than over-testing a healthy one (FP).

Recall (Sensitivity)

- **Definition:** Out of all instances that were **actually Positive**, how many did the model **correctly predict**?
- **Formula:** $\text{Recall} = \frac{TP}{TP+FN}$
- **When to Optimize:** When the cost of a **False Negative** is very high.
- **Example:** Disease outbreak prediction, cancer screening, or fraud detection. Missing a positive case is catastrophic.

The Trade-off

Increasing Recall usually decreases Precision, and vice versa. You cannot maximize both simultaneously without improving the underlying model.

The F1-Score

- **Definition:** The **harmonic mean** of Precision and Recall.
- **Formula:** $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **Why Harmonic Mean?** It penalizes extreme values. If either Precision or Recall is very low, the F1-Score will be low. (An arithmetic mean would misleadingly mask a zero in one metric).
- **When to Optimize:** When you need a single metric to balance both False Positives and False Negatives, especially in imbalanced datasets.

Part 4: Hands-on Python Lab (Evening Session)

"Applying theory to the Heart Disease Dataset."

Lab Setup: Dataset & Dependencies

- **Dataset:** `Heart.csv` (Clinical measurements for heart disease prediction).
- **Target Variable:** HD (0 = No Heart Disease, 1 = Heart Disease).
- **Features:** Age, Sex, ChestPain, RestBP, Chol, Fbs, RestECG, MaxHR, ExAng, Oldpeak, Slope, Ca, Thal.
- **Dependencies:**
 - Data Manipulation: `pandas`, `numpy`
 - Machine Learning: `scikit-learn`
 - Visualization: `matplotlib`, `seaborn`

4.1 Loading the Data

- We load the dataset and immediately inspect for missing values and data types.
- Note: ChestPain and Thal are categorical, while HD is our binary target.

Python Implementation

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 sns.set_theme(style="whitegrid")
7 plt.rcParams['figure.figsize'] = (10, 6)
8
9 # Load the Heart Disease dataset
10 df = pd.read_csv('Heart.csv')
11
12 # Quick inspection
13 print(df.info())
14 print(df['HD'].value_counts(normalize=True))
15
```


4.2 EDA & Target Distribution

- Checking the class balance of the target variable HD is critical before choosing evaluation metrics.
- Visualizing the distribution of key features like Age and MaxHR by target class.

Python Implementation

```
1 # Check class balance
2 print(df['HD'].value_counts())
3
4 # Visualize Age distribution by Heart Disease status
5 sns.histplot(data=df, x='Age', hue='HD', multiple='stack', bins=20)
6 plt.title('Age Distribution by Heart Disease Status')
7 plt.show()
8
```

4.3 Preprocessing Strategy

- We must encode categorical variables and scale numeric variables (especially crucial for k-NN and Logistic Regression).

Python Implementation

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.preprocessing import StandardScaler, OneHotEncoder
3
4 numeric_features = ['Age', 'RestBP', 'Chol', 'MaxHR', 'Oldpeak', 'Ca']
5 categorical_features = ['Sex', 'ChestPain', 'Fbs', 'RestECG', 'ExAng', 'Slope', 'Thal',
6                        '']
7
8 preprocessor = ColumnTransformer(
9     transformers=[
10         ('num', StandardScaler(), numeric_features),
11         ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
12         categorical_features)
13     ])
14
```

4.4 Train / Test Split

- We separate features (X) and target (y), then split into 80% training and 20% testing.
- We use `stratify=y` to ensure the train and test sets maintain the same proportion of Heart Disease cases as the original dataset.

Python Implementation

```
1 from sklearn.model_selection import train_test_split
2
3 X = df.drop(columns=['AHD', 'HD']) # Drop both string and numeric targets
4 y = df['HD'] # Use numeric 0/1 target
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.20, random_state=42, stratify=y
8 )
9
```

4.5 Building the Logistic Regression Baseline

- We bundle the preprocessor and the Logistic Regression model into a single Pipeline to prevent data leakage.

Python Implementation

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.linear_model import LogisticRegression
3
4 logreg_pipe = Pipeline(steps=[
5     ('preprocessor', preprocessor),
6     ('classifier', LogisticRegression(random_state=42, max_iter=1000))
7 ])
8
9 # Fit the model on training data
10 logreg_pipe.fit(X_train, y_train)
11
```

4.6 Evaluating the Logistic Regression Baseline

- Generate predictions and calculate the foundational metrics.

Python Implementation

```
1 from sklearn.metrics import accuracy_score, classification_report
2
3 y_pred_logreg = logreg_pipe.predict(X_test)
4
5 print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_logreg))
6 print("\nClassification Report:\n",
7       classification_report(y_test, y_pred_logreg, target_names=['No HD (0)', 'HD (1)'],
8                             ']))
```

4.7 Implementing the k-NN Classifier

- We build a second pipeline for k-NN. The StandardScaler in our preprocessor is now critically important.

Python Implementation

```
1 from sklearn.neighbors import KNeighborsClassifier
2
3 knn_pipe = Pipeline(steps=[
4     ('preprocessor', preprocessor),
5     ('classifier', KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)) # p
6     =2 is Euclidean
7 ])
8
9 knn_pipe.fit(X_train, y_train)
10 y_pred_knn = knn_pipe.predict(X_test)
```

4.8 Comparing Distance Metrics in k-NN

- We can easily swap the distance metric in scikit-learn to observe performance changes.

Python Implementation

```
1 # Euclidean Distance (p=2)
2 knn_euclidean = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
3 # Manhattan Distance (p=1)
4 knn_manhattan = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=1)
5
6 # Note: In a full pipeline, you would update the pipeline step or create new
7 #       pipelines
8 # For demonstration, we evaluate the default (Euclidean) vs a custom Manhattan setup
```

4.9 Generating the Confusion Matrix

- Visualizing the Confusion Matrix provides immediate insight into the types of errors the model is making.

Python Implementation

```
1 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
2
3 # For Logistic Regression
4 cm_logreg = confusion_matrix(y_test, y_pred_logreg)
5 disp = ConfusionMatrixDisplay(confusion_matrix=cm_logreg,
6                               display_labels=['No HD (0)', 'HD (1)'])
7 disp.plot(cmap='Blues')
8 plt.title('Logistic Regression Confusion Matrix')
9 plt.show()
10
```


4.10 Calculating Precision, Recall, and F1

- While `classification_report` shows these, we can also extract them individually for reporting.

Python Implementation

```
1 from sklearn.metrics import precision_score, recall_score, f1_score
2
3 # Evaluate for the Positive Class (HD = 1)
4 precision = precision_score(y_test, y_pred_logreg, pos_label=1)
5 recall = recall_score(y_test, y_pred_logreg, pos_label=1)
6 f1 = f1_score(y_test, y_pred_logreg, pos_label=1)
7
8 print(f"Precision: {precision:.3f}")
9 print(f"Recall:      {recall:.3f}")
10 print(f"F1-Score:   {f1:.3f}")
11
```

4.11 Model Comparison: LogReg vs. k-NN

Model	Accuracy	Precision (HD=1)	Recall (HD=1)	F1-Score
Logistic Regression	0.85	0.83	0.88	0.85
k-NN (k=5, Euclidean)	0.75	0.70	0.80	0.75

Observation

Logistic Regression typically outperforms basic k-NN on this dataset due to its ability to weigh features appropriately, whereas k-NN treats all scaled features equally and is sensitive to local noise. (*Note: Exact values depend on the random split, but the trend holds*).

Part 5: Deliverables & Next Steps

"Consolidating today's learning."

Deliverable: Baseline Model Evaluation Report

Evening Lab Submission Requirement

Participants must submit a concise Jupyter Notebook or PDF report containing:

- 1 **Data Summary:** Brief description of the `Heart.csv` dataset and class balance.
- 2 **Preprocessing Steps:** Justification for scaling and encoding choices.
- 3 **Model Outputs:** Confusion Matrices and Classification Reports for both Logistic Regression and k-NN.
- 4 **Comparative Analysis:** A short paragraph (3-4 sentences) comparing the two models, explicitly discussing the Precision/Recall trade-off in the context of Kenyan healthcare triage.

Key Takeaways from Day 1

- **Classification** predicts categories, using probability thresholds to make discrete decisions.
- **Baseline models** (Logistic Regression, k-NN) are essential first steps before deploying complex algorithms.
- **Accuracy is deceptive** in imbalanced datasets; always inspect the Confusion Matrix.
- **Precision vs. Recall** is a business/policy decision: In healthcare, we often prioritize Recall to minimize False Negatives (missed diagnoses).
- **Feature scaling** is non-negotiable for distance-based algorithms like k-NN.

Preview of Day 2: Advanced Tree-Based Models

Tomorrow's Agenda

- **Morning:** Decision Trees, Random Forests, and XGBoost theory. Understanding Gini Impurity and Ensemble learning (Bagging vs. Boosting).
- **Evening Lab:** Upgrading our Heart Disease pipeline with Random Forest and XGBoost, including hyperparameter tuning via GridSearchCV.
- *Preparation:* Ensure your Python environment has `xgboost` and `scikit-learn` installed.

Questions?

- Clarifications on Logistic Regression vs. k-NN?
- Questions on interpreting the Confusion Matrix?
- Challenges with the evening Python lab setup?

"The goal is not just to build models, but to build models that make safe, interpretable, and impactful decisions."

Thank You!

Thank You!

Victor Wandera Lumumba

Statistician / Data Analyst / ML Expert

Mediacrest Training College

Nairobi, Kenya

Email: [lumumbavictor172@gmail.com](mailto:lumbavictor172@gmail.com)

Web: beyonddataanalytics.online

GitHub: [Lumumba1992](https://github.com/Lumumba1992)